МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Лабораторная работа № 7

по дисциплине «Математические основы баз данных»

по теме: «Итоговый проект»

Выполнил студент группы:
БВТ2505
Пондяков Игорь Дмитриевич
Проверил:
Быватов Семён Юрьевич

Москва 2026

Цель работы:

Интегрировать все технологии, изученные в рамках курса (проектирование, SQL, оконные функции, PL/pgSQL, JSONB, оптимизацию), в единый законченный проект базы данных. Продемонстрировать умение самостоятельно спроектировать, реализовать, наполнить данными, оптимизировать и документировать реляционную базу данных для реальной предметной области «Интернет-провайдер».

Выполнение

1. Завершённая схема базы данных (3НФ)

Разработана и реализована нормализованная схема БД провайдера, включающая 9 основных сущностей: subscribers, contracts, tariffs, payments, equipment, support_tickets, employees, services, contract_services.

Все таблицы приведены к 3НФ.

Реализованы первичные ключи (GENERATED ALWAYS AS IDENTITY), внешние ключи с правилами ON DELETE RESTRICT/CASCADE, ограничения CHECK, NOT NULL, UNIQUE.

Иерархия категорий услуг смоделирована через самоссылку (parent_id).

Связь «многие-ко-многим» между договорами и услугами реализована через ассоциативную таблицу contract_services.

2. Наполнение данными

База наполнена согласованными тестовыми данными, достаточными для содержательной аналитики:

subscribers: 1000 абонентов.

contracts: 1000 договоров (80% active, 10% suspended, 10% closed).

payments: ~3000 платежей.

equipment: 1500 единиц оборудования.

support_tickets: 500 заявок.

3. Аналитические запросы

Реализован набор сложных SQL-запросов:

Многотабличные выборки с INNER JOIN (3+ таблицы) и LEFT JOIN.

Агрегация с GROUP BY и фильтрация групп через HAVING.

Скалярные подзапросы, предикаты IN, коррелированные подзапросы с EXISTS.

Общие табличные выражения (CTE), включая рекурсивный CTE для обхода дерева категорий услуг.

4. Оконные функции

Проведена многомерная аналитика без схлопывания строк:

Ранжирование тарифов (ROW_NUMBER, RANK, DENSE_RANK).

Выборка Top-N дорогих тарифов в каждой категории подключения.

Нарастающий итог выручки по дням (SUM() OVER(ORDER BY ...)).

Доля выручки тарифа в общем объеме по типу подключения (SUM() OVER(PARTITION BY ...)).

Сравнение ежедневной выручки с соседними днями (LAG, LEAD).

5. Серверная логика и целостность

Транзакции: Реализованы сценарии смены тарифа с COMMIT, полным ROLLBACK при ошибке и частичным откатом через SAVEPOINT/блок EXCEPTION.

Функции PL/pgSQL: Создана функция fn_connect_service для атомарного подключения услуги с проверками бизнес-правил и функцией fn_revenue_for_period для расчёта сводных показателей.

Триггеры:

trg_audit_contracts: автоматический аудит изменений договоров в таблицу audit_log (JSONB).

trg_recalc_cost: автоматический пересчёт денормализованного поля total_monthly_cost при изменении подключённых услуг.

6. Современные возможности PostgreSQL (JSONB + Индексы)

К таблице equipment добавлен столбец specs JSONB для хранения гибких характеристик (Wi-Fi стандарт, порты, ОС).

Реализованы запросы с операторами ->>, @> для фильтрации оборудования.

Создан GIN-индекс idx_equipment_specs_gin для ускорения поиска по JSONB.

Добавлены B-tree индексы на contracts(start_date) и contracts(subscriber_id).

Подтверждено снижение времени выполнения запросов и изменение плана с Seq Scan на Index Scan / Bitmap Index Scan.

7. Представления (VIEW)

Созданы два аналитических представления для типовых отчётов:

v_tariff_revenue: сводка по выручке и количеству активных договоров каждого тарифа.

v_support_tickets_summary: статистика загруженности инженеров техподдержки с процентом выполненных заявок.

Единый листинг с разграничением по заданиям:

```
-- 1. ОЧИСТКА И СОЗДАНИЕ СХЕМЫ (DDL из ЛР1)
DROP TABLE IF EXISTS audit_log CASCADE;
DROP TABLE IF EXISTS support_tickets CASCADE;
DROP TABLE IF EXISTS equipment CASCADE;
DROP TABLE IF EXISTS payments CASCADE;
DROP TABLE IF EXISTS contract_services CASCADE;
DROP TABLE IF EXISTS contracts CASCADE;
DROP TABLE IF EXISTS subscribers CASCADE;
DROP TABLE IF EXISTS tariffs CASCADE;
DROP TABLE IF EXISTS services CASCADE;
DROP TABLE IF EXISTS service_categories CASCADE;
DROP TABLE IF EXISTS employees CASCADE;

CREATE TABLE service_categories (
   category_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
   parent_id BIGINT REFERENCES service_categories(category_id) ON
DELETE RESTRICT,
   name TEXT NOT NULL UNIQUE
);

CREATE TABLE tariffs (
   tariff_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   name TEXT NOT NULL UNIQUE,
   speed_down INT NOT NULL CHECK(speed_down > 0),
   speed_up INT NOT NULL CHECK(speed_up > 0),
   monthly_price NUMERIC(10,2) NOT NULL CHECK(monthly_price > 0),
   connection_type TEXT NOT NULL CHECK(connection_type IN ('Оптика',
'Ethernet'))
);

CREATE TABLE employees (
   employee_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
   full_name TEXT NOT NULL,
   position TEXT NOT NULL,
   phone TEXT NOT NULL UNIQUE
);
```

```
CREATE TABLE services (
    service_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_id BIGINT NOT NULL REFERENCES
service_categories(category_id),
    name TEXT NOT NULL UNIQUE,
    monthly_price NUMERIC(10,2) NOT NULL CHECK(monthly_price > 0)
);

CREATE TABLE subscribers (
    subscriber_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    full_name TEXT NOT NULL,
    passport TEXT NOT NULL UNIQUE,
    phone TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL UNIQUE
);

CREATE TABLE contracts (
    contract_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    subscriber_id BIGINT NOT NULL REFERENCES subscribers(subscriber_id)
ON DELETE RESTRICT,
    tariff_id BIGINT NOT NULL REFERENCES tariffs(tariff_id),
    start_date DATE NOT NULL DEFAULT CURRENT_DATE,
    status TEXT NOT NULL CHECK(status IN ('active', 'suspended', 'closed')),
    total_monthly_cost NUMERIC(10,2) DEFAULT 0 -- Денормализованное
поле из ЛР5
);

CREATE TABLE contract_services (
    contract_id BIGINT NOT NULL REFERENCES contracts(contract_id) ON
DELETE CASCADE,
    service_id BIGINT NOT NULL REFERENCES services(service_id) ON
DELETE RESTRICT,
    connected_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (contract_id, service_id)
);

CREATE TABLE payments (
    payment_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    contract_id BIGINT NOT NULL REFERENCES contracts(contract_id) ON
DELETE RESTRICT,
    amount NUMERIC(10,2) NOT NULL CHECK(amount > 0),
    payment_date TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
    payment_method TEXT NOT NULL
);

CREATE TABLE equipment (
    equipment_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    contract_id BIGINT NOT NULL REFERENCES contracts(contract_id) ON
DELETE CASCADE,
    type TEXT NOT NULL,
    model TEXT NOT NULL,
    serial_number TEXT NOT NULL UNIQUE,
    issue_date DATE NOT NULL,
    specs JSONB -- JSONB из ЛР6
);

CREATE TABLE support_tickets (
    ticket_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    contract_id BIGINT NOT NULL REFERENCES contracts(contract_id) ON
DELETE RESTRICT,
    employee_id BIGINT REFERENCES employees(employee_id),
    description TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    status TEXT NOT NULL CHECK(status IN ('new', 'in_progress', 'done'))
);

CREATE TABLE audit_log (
    audit_id SERIAL PRIMARY KEY,
    table_name VARCHAR(50) NOT NULL,
    record_id INTEGER NOT NULL,
    action_type VARCHAR(10) NOT NULL,
    changed_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    changed_by VARCHAR(50) DEFAULT current_user,
    old_values JSONB,
    new_values JSONB
);

-- 2. НАПОЛНЕНИЕ ДАННЫМИ (из ЛР2)
INSERT INTO service_categories (name, parent_id) VALUES
('Интернет', NULL), ('Телевидение', NULL), ('Безопасность', NULL);
INSERT INTO service_categories (name, parent_id)
SELECT 'Оптика', category_id FROM service_categories WHERE name =
'Интернет';
INSERT INTO service_categories (name, parent_id)
SELECT 'HD Каналы', category_id FROM service_categories WHERE name =
'Телевидение';
```

```
INSERT INTO tariffs (name, speed_down, speed_up, monthly_price,
connection_type) VALUES
('Старт', 50, 20, 450.00, 'Оптика'),
('Домашний', 100, 50, 600.00, 'Оптика'),
('Геймер', 300, 300, 900.00, 'Оптика'),
('Бизнес', 500, 500, 1500.00, 'Ethernet');

INSERT INTO employees (full_name, position, phone) VALUES
('Иванов И.И.', 'Инженер', '+79001112233'),
('Петрова А.С.', 'Оператор', '+79004445566'),
('Сидоров В.В.', 'Менеджер', '+79007778899');

INSERT INTO services (category_id, name, monthly_price)
SELECT category_id, 'Антивирус', 100.00 FROM service_categories WHERE
name = 'Безопасность';
INSERT INTO services (category_id, name, monthly_price)
SELECT category_id, 'Статический IP', 150.00 FROM service_categories
WHERE name = 'Интернет';
INSERT INTO services (category_id, name, monthly_price)
SELECT category_id, 'ТВ Пакет Спорт', 200.00 FROM service_categories
WHERE name = 'HD Каналы';

INSERT INTO subscribers (full_name, passport, phone, email)
SELECT 'Абонент ' || g, '1234' || LPAD(g::text, 6, '0'),
    '+7900' || LPAD((g % 10000)::text, 4, '0') || LPAD((g / 10000)::text, 2, '0'),
    'user' || g || '@example.com'
FROM generate_series(1, 1000) AS g;

INSERT INTO contracts (subscriber_id, tariff_id, start_date, status)
SELECT s.subscriber_id, (random()*3+1)::int,
    CURRENT_DATE - (random()*365)::int,
    CASE WHEN random() < 0.8 THEN 'active'
        WHEN random() < 0.9 THEN 'suspended'
        ELSE 'closed' END
FROM subscribers s;

INSERT INTO payments (contract_id, amount, payment_date, payment_method)
SELECT c.contract_id, t.monthly_price,
    c.start_date + (n * interval '1 month'),
    CASE WHEN n % 2 = 0 THEN 'Банковская карта' ELSE 'Наличные' END
FROM contracts c JOIN tariffs t ON c.tariff_id = t.tariff_id
CROSS JOIN LATERAL generate_series(1, 3) AS n
WHERE c.status != 'closed';
```

```
INSERT INTO equipment (contract_id, type, model, serial_number, issue_date,
specs)
SELECT c.contract_id,
      CASE WHEN random() < 0.7 THEN 'Роутер' ELSE 'ТВ-приставка' END,
      CASE WHEN random() < 0.7 THEN 'TP-Link Archer' ELSE 'MAG 250'
END,
      'SN-' || LPAD((row_number() OVER ())::text, 6, '0'),
      c.start_date,
      CASE WHEN random() < 0.7 THEN '{"wifi_standard": "Wi-Fi 6", "ports":
4}'::jsonb
           ELSE '{"resolution": "4K", "os": "Android TV"}'::jsonb END
FROM contracts c WHERE c.status = 'active' LIMIT 1500;

INSERT INTO support_tickets (contract_id, employee_id, description,
created_at, status)
SELECT c.contract_id, (random()*2+1)::int,
      CASE WHEN random() < 0.3 THEN 'Нет интернета'
           WHEN random() < 0.6 THEN 'Медленная скорость'
           ELSE 'Настройка ТВ' END,
      c.start_date + (random()*100)::int * interval '1 day',
      CASE WHEN random() < 0.5 THEN 'new'
           WHEN random() < 0.8 THEN 'in_progress'
           ELSE 'done' END
FROM contracts c WHERE c.status = 'active' LIMIT 500;

INSERT INTO contract_services (contract_id, service_id, connected_at)
SELECT c.contract_id, s.service_id, c.start_date
FROM contracts c CROSS JOIN services s
WHERE random() < 0.2 AND c.status = 'active';

-- 3. ОБНОВЛЕНИЕ ДЕНОРМАЛИЗОВАННОГО ПОЛЯ (из ЛР5)
UPDATE contracts c SET total_monthly_cost = t.monthly_price + COALESCE(
   (SELECT SUM(s.monthly_price) FROM contract_services cs
    JOIN services s ON cs.service_id = s.service_id
    WHERE cs.contract_id = c.contract_id), 0)
FROM tariffs t WHERE c.tariff_id = t.tariff_id;

-- 4. ИНДЕКСЫ (из ЛР6)
CREATE INDEX idx_contracts_subscriber ON contracts(subscriber_id);
CREATE INDEX idx_contracts_start_date ON contracts(start_date);
CREATE INDEX idx_equipment_specs_gin ON equipment USING GIN
(specs);

-- 5. ФУНКЦИИ И ТРИГГЕРЫ (из ЛР5)
```

```
CREATE OR REPLACE FUNCTION fn_audit_contracts() RETURNS
TRIGGER AS $$
BEGIN
   IF TG_OP = 'UPDATE' THEN
      INSERT INTO audit_log (table_name, record_id, action_type, old_values,
new_values)
      VALUES (TG_TABLE_NAME, OLD.contract_id, TG_OP, to_jsonb(OLD),
to_jsonb(NEW));
      RETURN NEW;
   ELSIF TG_OP = 'INSERT' THEN
      INSERT INTO audit_log (table_name, record_id, action_type, new_values)
      VALUES (TG_TABLE_NAME, NEW.contract_id, TG_OP, NULL,
to_jsonb(NEW));
      RETURN NEW;
   ELSIF TG_OP = 'DELETE' THEN
      INSERT INTO audit_log (table_name, record_id, action_type, old_values)
      VALUES (TG_TABLE_NAME, OLD.contract_id, TG_OP, to_jsonb(OLD),
NULL);
      RETURN OLD;
   END IF;
   RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_contracts AFTER INSERT OR UPDATE OR
DELETE ON contracts
FOR EACH ROW EXECUTE FUNCTION fn_audit_contracts();

CREATE OR REPLACE FUNCTION fn_recalc_contract_cost() RETURNS
TRIGGER AS $$
DECLARE v_contract_id INTEGER; v_tariff_price NUMERIC(10,2);
v_services_sum NUMERIC(10,2);
BEGIN
   v_contract_id := COALESCE(NEW.contract_id, OLD.contract_id);
   SELECT t.monthly_price INTO v_tariff_price FROM contracts c
   JOIN tariffs t ON c.tariff_id = t.tariff_id WHERE c.contract_id =
v_contract_id;
   SELECT COALESCE(SUM(s.monthly_price), 0) INTO v_services_sum
   FROM contract_services cs JOIN services s ON cs.service_id = s.service_id
   WHERE cs.contract_id = v_contract_id;
   UPDATE contracts SET total_monthly_cost = v_tariff_price +
v_services_sum
   WHERE contract_id = v_contract_id;
   RETURN COALESCE(NEW, OLD);
END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_recalc_cost AFTER INSERT OR UPDATE OR
DELETE ON contract_services
FOR EACH ROW EXECUTE FUNCTION fn_recalc_contract_cost();

-- 6. ПРЕДСТАВЛЕНИЯ (из ЛР3)
CREATE OR REPLACE VIEW v_tariff_revenue AS
SELECT t.tariff_id, t.name AS tariff_name, t.monthly_price,
    COUNT(c.contract_id) AS active_contracts,
    ROUND(SUM(t.monthly_price), 2) AS total_monthly_revenue
FROM tariffs t JOIN contracts c ON t.tariff_id = c.tariff_id
WHERE c.status = 'active'
GROUP BY t.tariff_id, t.name, t.monthly_price;

CREATE OR REPLACE VIEW v_support_tickets_summary AS
SELECT e.employee_id, e.full_name AS employee_name, e.position,
    COUNT(st.ticket_id) AS total_tickets,
    COUNT(st.ticket_id) FILTER (WHERE st.status = 'done') AS done_tickets,
    ROUND(100.0 * COUNT(st.ticket_id) FILTER (WHERE st.status = 'done')
        / NULLIF(COUNT(st.ticket_id), 0), 1) AS completion_rate_percent
FROM employees e LEFT JOIN support_tickets st ON e.employee_id =
st.employee_id
GROUP BY e.employee_id, e.full_name, e.position;
```

Вывод

В ходе выполнения итоговой лабораторной работы была успешно разработана и оптимизирована база данных интернет-провайдера, объединяющая все компетенции курса. Проект демонстрирует полный цикл разработки: от ER-моделирования и нормализации до внедрения серверной логики, аналитики и оптимизации производительности. Принятые архитектурные решения (разделение логики между БД и приложением, осознанное использование индексов и JSONB, применение триггеров для поддержания целостности) обоснованы требованиями предметной области и подтверждены результатами анализа планов выполнения. База готова к использованию в качестве прототипа информационной системы провайдера.